

1.Number of days between two dates

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

class Solution {
    public int daysBetweenDates(String date1, String date2) {

        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);

        return (int) Math.abs(ChronoUnit.DAYS.between(d1,d2));
    }
}
```

2.Day of the year

```
import java.time.LocalDate;

class Solution {
    public int dayOfYear(String date) {
        return LocalDate.parse(date).getDayOfYear();
    }
}
```

3.Day of the week

```
class Solution {
    public String dayOfTheWeek(int day, int month, int year) {
        String[] days = {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday"};
        int[] daysInMonth = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        int totalDays = 0;
        for(int y = 1971; y < year; y++){
            totalDays += leapYear(y) ? 366 : 365;
        }
        for(int m = 1; m < month; m++){
            if(m == 2 && leapYear(year)) totalDays += 29;
            else totalDays += daysInMonth[m - 1];
        }
        totalDays += (day - 1);
        int dayIndex = (5 + totalDays) % 7;
        return days[dayIndex];
    }
    private boolean leapYear(int year){
```

```

        return (year % 400 == 0 || year % 4 == 0 && year % 100 != 0);
    }
}

```

4.Largest Number

```

class Solution {
    public String largestNumber(int[] nums) {
        // Convert integers to strings
        String[] store = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            store[i] = String.valueOf(nums[i]);
        }

        // Custom sort with comparator
        Arrays.sort(store, (a, b) -> {
            String order1 = a + b;
            String order2 = b + a;
            return order2.compareTo(order1); // descending order
        });

        // ♦ handle all zeros case
        if (store[0].equals("0")) return "0";

        // Concatenate all strings
        StringBuilder ans = new StringBuilder();
        for (String s : store) {
            ans.append(s);
        }

        return ans.toString();
    }
}

```

5.Sort the People

```

class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        int n = names.length;
        Map<Integer, String> mapping = new HashMap<>();

        for (int i = 0; i < n; ++i) {
            mapping.put(heights[i], names[i]);
        }

        Arrays.sort(heights);
    }
}

```

```
    for (int i = 0; i < n / 2; ++i) {  
        int temp = heights[i];  
        heights[i] = heights[n - 1 - i];  
        heights[n - 1 - i] = temp;  
    }  
  
    for (int i = 0; i < n; ++i) {  
        names[i] = mapping.get(heights[i]);  
    }  
  
    return names;  
}  
}
```